



VRIJE
UNIVERSITEIT
BRUSSEL



Bachelor Thesis

CHORD DETECTION USING DISCRETE RESONANCE SPECTROGRAM

Ebubekir Kahriman

May 31, 2026

Promotor: Prof. Geraint Wiggins
Artificiële Intelligentie

Contents

1	Introduction	2
2	Literature Study	2
2.1	The Information Dynamics of Thinking (IDyOT)	2
2.2	Spectral Knowledge Representation	3
2.2.1	Sound in the Spectral Domain: Fourier and Resonances	3
2.2.2	The Mathematics of the Hilbert Space	5
2.2.3	Discrete Resonance Spectrogram (DRS)	5
2.3	Fundamentals of Music Theory	6
3	Development of the Similarity Metric	6
3.1	The Baseline Approach: Normalized Inner Product (Cosine Similarity)	7
3.1.1	The Phase Rotation Problem	8
3.2	Spectral Line Shape Approach	8
3.2.1	The Lorentzian Factor and Visual Overlap	8
3.2.2	Adding Volume as a Weight	8
3.2.3	Fixed Height of Peak	8
3.2.4	The Normalization Failure	9
3.3	The Normalized Lorentzian approach	9
3.3.1	Formula	9
3.3.2	Removing factor $\frac{1}{\pi}$	9
3.3.3	Adding Power as a Weight	10
3.3.4	Raw Overlap and Double Factor Problem	10
3.3.5	The Convolution Solution	10
3.3.6	Cauchy-Schwarz Normalization	11
4	Experimental Setup	11
4.1	Dataset	12
4.1.1	Synthetic Audio	12
4.1.2	Real Audio	12
4.2	Evaluation Metrics	12
5	Results and Analysis	13
5.1	Finding Real Changes	13
5.2	The Limitation of the Phase Method	13
5.3	The Failed Unnormalized Lorentzian Method	14
5.4	The Working Method	15
6	Final F1-Score Results	16
6.1	Syntethic Data	16
6.2	Monophonic Sound	16
6.3	Homophonic Sound	16
6.4	Polyphonic Sound	17
7	Conclusion	17

1 Introduction

Music is full of complex overlapping sounds. A human ear easily hears when a song changes. A computer finds this exact task harder. Software often struggles to understand chaotic acoustic audio signals. Standard audio programs usually model sound with the traditional Fourier Transform. We try a different approach in this thesis. We use an existing technique called Discrete Resonance Spectrum (DRS) introduced by Homer et al. (2024). The DRS is comparable to the Fourier Transform because it also models sound. The internal technique is however completely different. We will fully explain how DRS works later in our literature study.

We use the DRS to solve our main problem. We want to build a reliable algorithm that detects chord changes automatically. We will also explain the exact definition of a chord later. First we explain the basic theory behind audio and sound waves. Then we test three different mathematical approaches on our DRS data. We show why the first two methods are not suitable for our project. Although these methods are not naturally incorrect, they were designed for a different purpose and are therefore not applicable in our context. Finally we present our successful normalized Cauchy distribution method. We test this winning program on real world music. This final test proves the academic value of our project.

2 Literature Study

Before we start explaining our project, this chapter will give the theoretical background of the project. In this literature study we will discuss the important concepts that are needed for our chord change detection. The next sections talk about the cognitive architecture that is the base of this work, and explains how the brain processes sound. Furthermore, we look to the mathematical representation of sound, where we compare the normal Fourier transform with our resonances (DRS). This chapter also introduces the mathematical space that is required for our calculations, together with the Discrete Resonance Spectrogram (DRS) software. Finally, a short overview of basic music theory is included to give some context about the chord structures and tones.

2.1 The Information Dynamics of Thinking (IDyOT)

To understand how we can automatically detect chord changes in an audio signal, we first need to look at how human perception can be modelled. A cognitive architecture is as a blueprint of the brain. It models how human thinking and learning work as one complete system (Wiggins, 2020). In this project, we use the framework of the Information Dynamics of Thinking (IDyOT). Compared to many traditional cognitive architectures, IDyOT is a "bottom-up" (Wiggins, 2020). This means that the system does not get any programmed, "top-down" rules. For example, we do not give the system a music theory book that explicitly tells it the rules of a C-major chord. Instead, the system is built on basic mathematical structures, specifically waves and oscillations (Wiggins, 2020). The goal of IDyOT is to explain how human perception works and to show that complex behavior, such as pattern recognition, appears naturally from the simple math of interacting waves (Wiggins, 2020).

For this project, we will focus on the auditory part of IDyOT architecture. In this model, knowledge is not stored as symbols or another way of storage of data. Instead, information is encoded in the brain as vibrations, or also called *oscillations*. When a new sound enters the ear, it is also a wave. If this new incoming wave is similar to the stored knowledge in the brain, the neural structures start to vibrate with it. This is called constructive interference which we will talk about later.

When a chord plays continuously, the incoming sound will resonates nicely with the current state of the system. However, when the chord changes, the new sound waves no longer match the earlier vibrations (Homer et al., 2024). The waves fight, the resonance stops, and destructive interference (Ockenga et al., 2023). In our software model, this clash/fight causes a drop in the similarity score. When tracking these drops over time, we can exactly point out where the chord transitions happen.

2.2 Spectral Knowledge Representation

To analyse sound like how the human brain is doing it, we need a specific format to store the audio information in our computer model. The way how a system is storing information is called Knowledge Representation. Traditional models are often using top-down rules or symbols to represent the music. But in this project, we are using a approach called *Spectral Knowledge Representation* (Homer et al., 2024).

The main idea of this Spectral Knowledge Representation is that the knowledge is stored as literal representations of waves, and more specifically *resonances* (Homer et al., 2024). This method is heavily inspired on the biological brain. In the brain, neurons process and remember information by using electrical vibrations and neural oscillations (Homer et al., 2024). Because our goal is to model the human music perception, it is much more logical to represent our audio data as interacting waves instead of symbols.

In the next subsections, we will explain exactly how this representation works. We will discuss why we use decaying resonances instead of the standard Fourier transform. Furthermore, we look to the mathematical space that is used to calculate the similarity between those waves. Finally, we will explain the software that translates real audio into this spectral representation.

2.2.1 Sound in the Spectral Domain: Fourier and Resonances

To analyse an audio signal in a computer, the sound must be translated into a mathematical format. In standard signal processing, the most popular method for this is the Fourier transform. The Fourier transform breaks a sound down in a big sum of infinite sine and cosine waves (noa, 2016).

We first need to look at the basic physics of sound. Multiple sounds interact with each other when they overlap. This behavior is ruled by the *Principle of Superposition* (Ockenga et al., 2023). The principles of wave physics state that “if two waves interfere the amplitude of the resulting light wave will be equal to the vector sum of the amplitudes of the two interfering waves” (Ockenga et al., 2023). While this specific quote mentions light the underlying *Principle of Superposition* is universal for all linear systems. It applies perfectly to the acoustic audio waves that form musical chords.

However, for a cognitive model that wants to act like a real brain, Fourier is not the most logical approach (Homer et al., 2024).

The main problem is how the standard Fourier transform is handling sounds. To model a short sound event, the Fourier transform needs to use thousands of waves with all different frequencies. Exactly on the moment when the sound happens, all these different phases must align perfectly with each other. The aligned waves will make the resulted wave have a higher amplitude what result in create a big peak in the sound, which is called constructive interference (Ockenga et al., 2023).

But the biggest issue occurs during the silence before and after that sound. Because the standard Fourier waves are infinite, they are never stopping. During the silence, all these thousands of waves are actually still running in the background. But their phases cross each other in such

a specific way that they cancel the sound out perfectly to zero. This process is called destructive interference (Ockenga et al., 2023). The resultant wave will have a lower amplitude because the waves are out of phase. Simply put, they are going to cancel each other out. Because it needs infinite waves just to simulate silence, this creates a very messy mathematical representation. It is definitely not how human ears or physical instruments are working (Homer et al., 2024).

Instead of using these infinite waves, this project uses resonances, which are also known in the literature as *damped oscillators* (Homer et al., 2024). You can think of a resonance like a sleeping string of a physical guitar. The string is just resting and doing nothing, until a specific frequency hits it. Only then, it wakes up and starts vibrating along with the sound. To use this biological concept in a computer, damped oscillator are defined mathematically. In the spectral model, every single resonance is constructed using exactly four parameters (Homer et al., 2024):

- **Frequency:** This determines the pitch or the speed of the vibration (measured in Hertz (Hz)).
- **Amplitude:** This is the start strength of the vibration at the exact moment it starts. A harder hit on the string results in a higher amplitude. Also called the *volume*.
- **Phase:** This represents the exact starting position or angle of the wave.
- **Decay:** This defines how fast the energy of the vibration disappears over time (damping). A high decay means the sound dies out very quickly, while a low decay means the string keeps ringing for a longer time.

Two Ways of Interpretation

It is very important to understand that the resonance has two different ways of representation, as shown in Figure 1. In the time domain, it looks like the vibrating guitar string. But if we analyze this resonance in the frequency domain, the mathematics show that it takes the shape of a *Lorentzian peak* (Homer et al., 2024). The frequency, amplitude, and decay parameters directly determine the exact location, height, and width of this peak. This peak will play an important role later on in the project.

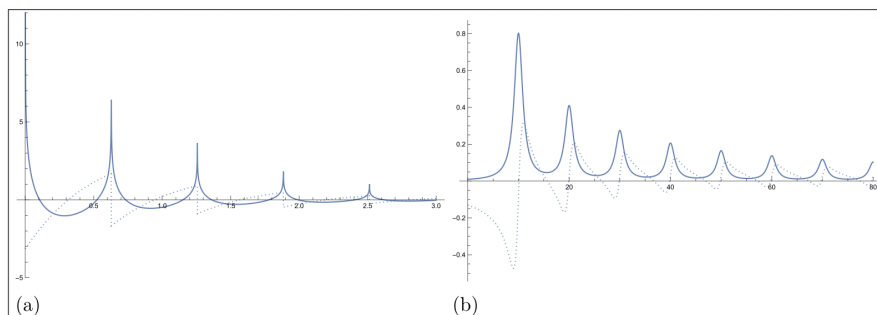


Figure 1: A harmonic resonance represented in both the time domain (a) and the frequency domain (b). Image from Homer et al. (2024), licensed under CC BY 4.0.

When the incoming sound stops, the vibration of this mathematical string is naturally dying out over time because of this decay parameter. It does not continue infinitely to cancel other waves out. In the real world, every sound eventually dies out because of friction. Because these resonances fade out naturally, this is a much more biologically reasonable method. Furthermore, it makes it much easier for our algorithm to detect sudden changes in the audio, like chord transitions, without having the mathematical chaos of infinite waves.

2.2.2 The Mathematics of the Hilbert Space

We need a robust mathematical foundation before we can start thinking of mathematical ways to satisfy our research title. We cannot simply take two continuous waves and calculate a score out of nowhere. The data must exist inside a specific geometric system that supports mathematical rules for angles and distances. Homer et al. (2024), project the waves mathematically into a space called the *Hilbert space*.

A normal physical space only has an infinite amount of dimensions (X, Y, Z etc.). However, the Hilbert space that's used in this theory is an vector space with an infinite amount of dimension (Homer et al., 2024). In this specific space, every single frequency is represented as its own separate axis (Homer et al., 2024). When a resonance exists in this space, it is represented as a single vector. You can visualize this vector as an arrow that points in a specific direction. If a resonance contains a specific frequency, the vector will point along that specific axis. When a complex sound has multiple frequencies (for example the C chord with the C, E and G tone), the vector will trigger multiple axes at the same time. These activated axes form the fundamental building blocks of the knowledge representation.

The main theoretical reason to place the resonances inside this Hilbert space, is that it mathematically makes it possible to use the inner product (Homer et al., 2024):

$$\langle f, g \rangle.$$

The inner product calculates the angle between two vectors. If two vectors point in the same direction, they share the same frequency axes, and the inner product will be high (which theoretically points to constructive interference in our case). If they point in completely different directions, they are mathematically orthogonal, meaning the inner product is zero also called destructive interference. The direction of the vector indicates the tone(s) it consists of. The volume (amplitude) of the sound will determine the length of it. How this specific inner product calculation is practically applied to compare our sliding audio windows over time, will be explained in detail in Chapter 3.

The standard inner product has one important problem. It is heavily influenced by the length of the vectors. In the space we are working, this length represents the volume (amplitude) of the sound. We only want to compare the actual musical notes and not the volume. That is why we must normalize these vectors. We do this by using a method called *Cosine Similarity*. We divide the inner product by the lengths of both vectors. This completely removes the volume from our mathematics. It leaves us with a much stronger metric that looks at the direction of the vector. This guarantees that we only compare the true harmonic meaning of the sound (Homer et al., 2024).

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

2.2.3 Discrete Resonance Spectrogram (DRS)

To extract these specific resonances from real audio, we use a software called *Discrete Resonance Spectrogram* (DRS), developed by Homer et al. (2024). The DRS translates raw sound files into data that the cognitive model can understand.

Unlike standard Fourier analysis, which simply decomposes a signal into sines and cosines, the DRS models the audio signal as a rational function using a mathematical technique called the Padé approximant. The core of this process is finding the specific *poles* and *residues* of the audio signal (Homer et al., 2024). In the complex plane, the poles (ω_k) represent the resonant frequencies and their decay rates, while the residues (d_k) capture the initial amplitudes and

phases. The DRS extracts these complex parameters and translates them into the four real-valued spectral parameters: frequency (ϕ_k), decay (γ_k), amplitude (d_k), and phase (ψ_k).

Because a complex sound consists of multiple resonances that naturally fade out over time, its transformation into the frequency domain does not result in infinitely sharp lines. Instead, the mathematics order that the entire resonance spectrum is represented as a summation of multiple Lorentzian peaks (Homer et al., 2024).

$$f(\phi) = \frac{i}{\sqrt{2\pi}} \sum_{k=1}^K \frac{d_k}{\phi - \omega_k} = \frac{i}{\sqrt{2\pi}} \sum_{k=1}^K \frac{|d_k|e^{i\psi_k}}{\phi - (\phi_k + i\gamma_k)} \quad (1)$$

Every individual resonance (k) adds its own peak to this total spectrum. Because these peaks have a physical width caused by their decay, their acoustic energy bleeds into the nearby frequencies. In a complex sound, this physical width causes the multiple Lorentzian peaks within the summation to heavily overlap with each other. This overlapping will play an important role later in Chapter 3, where we going to focus on calculating the overlap and similarity of peaks and slices of sound.

2.3 Fundamentals of Music Theory

Before we can detect chord changes, we first need to explain the basic music theory used in this project. In the standard Western music theory, an octave is divided into 12 distinct tones (Joutsenvirta, 2005). These tones are:

C C# D D# E F F# G G# A A# B

Each tone has a specific frequency. The lowest C that is relevant for music starts around 16.35 Hz (Joutsenvirta, 2005), which is very close to the limit of human hearing, since humans can normally only hear sound waves between 20 Hz and 22,050 Hz (Brien, 2025). The distance between two neighbor tones is calculated with a fixed mathematical factor. To obtain the frequency of the next tone, for example when stepping from C to C#, the frequency must be multiplied by the factor $\sqrt[12]{2}$ (Müller, 2015). After 12 such steps, the note C is reached again, now one octave higher. At that point, the frequency has exactly doubled. So, from octave 1 to octave 2 for the same tone, you have to multiply with 2.

Furthermore, a chord is simply a combination of multiple tones played at the same time. For example, a basic C-major chord consists of three tones: C, E, and G. In music, you can play these chords in different ways by mixing tones from different octaves. For instance, the C and E may be taken from the fourth octave, while the G is taken from the fifth octave. This spreads the sound out and creates a different "voicing", but it still remains a C-major chord.

3 Development of the Similarity Metric

The goal in this project is to accurately detect changes in a audio file, more specific chord changes. To achieve this we need a mathematical formula that compares overlapping audio segments called *windows*. This chapter evaluates different mathematical approaches to calculate the spectral similarity and explains the reasoning behind the final proposed approach.

The literature defines an inner product approach that we used as a baseline (Homer et al., 2024). However this approach include the phase of a sound wave in the calculation because the main goal wherefore this formula was used was different. This will be a obstacle, more about this later in the next section (Homer et al., 2024).

Another approach models the peaks using the standard spectral line shape. This shape is mathematically represented by the basic Lorentzian function (Petrakis, 1967). This formula ignores phase information and forces the maximum height of every peak to exactly 1. This fixed height makes it very easy to visually compare different peaks on a computer screen. However it lacks the mathematical properties of a true probability distribution. Because the peak height is fixed independently of its width the total area under the curve loses its physical meaning. Without a stable area representing the true acoustic energy accurate mathematical normalization becomes impossible. This ultimately makes our similarity scores unreliable. We also explored several variants of this function which we will discuss later.

The proposed solution in this thesis uses the normalized form of the Lorentzian function or also called the Cauchy Distribution (Riley et al., 2006). Unlike the previous approach, this scientific version preserves the true area under the curve. Because this true area represents the actual energy of the sound, it allows for an correct calculation of the overlap between different peaks. This makes sure to have a correct mathematical normalization. By using the total power of the sound instead of the initial amplitude, this approach provides a stable algorithm for detecting chord transitions that is completely independent of phase.

3.1 The Baseline Approach: Normalized Inner Product (Cosine Similarity)

To set a baseline for our comparison we look at the mathematics developed by Homer et al. (2024). As we earlier mentioned in section 2.2 of the Literature, in the brain knowledge is stored as fading frequencies called damped oscillators instead of symbols. When a new sound wave actually matches the frequency and phase of a stored oscillator they activate each other. The mathematical formula used to calculate this physical overlap between two sound waves is the inner product. For two resonances the inner product is calculated using their complex amplitudes and complex frequencies (Homer et al., 2024):

$$\langle f|g \rangle = i \sum_j \sum_k \frac{d_j^* d_k}{\omega_j^* - \omega_k}$$

In this formula the top part contains the complex amplitudes which is actually the phase of the sound waves. The bottom part show the differences in frequency and decay. Homer workt this mathematical operation out to compare two completely separate audio files. When comparing these two files the inner product calculates the exact shared area between them. If the resonances in the first sound match with the resonances in the second audio the sound waves makes there amplitude stronger. This creates constructive interference and results in a large inner product value. If the resonances do not match, this means that the waves are cancelling each other out. This creates destructive interference and an inner product close to zero. The raw result of an inner product can be a very large number depending on the amplitude of the sound. To make this score useful we need normalization. The baseline method does this by converting the inner product into a cosine similarity. The algorithm divides the calculated overlap of the two audio files by the total power of each single file (Homer et al., 2024).

$$s_c(f, g) = \frac{Re[\langle f|g \rangle]}{\sqrt{\langle f|f \rangle \langle g|g \rangle}}$$

This normalization makes sure that the final result will always fall between 0.0 and 1.0. A score of 1.0 means that the two audio files contain identical and perfectly matched sounds. A score of 0.0 means that the audio files share no overlapping frequencies.

3.1.1 The Phase Rotation Problem

This baseline method is very good for comparing two separate sounds. However, it fails for our specific task. Our goal is not to compare two separate audio files. Instead we divide one audio file into slices (= windows) and compare these rolling windows over time. Because we move forward in time the phase of the sound wave naturally rotates. For example a sound wave in the first window can be at its highest point. A moment later in the second window that exact same sound wave might be at its lowest point. Musically the chord actually has not changed at all. But because the inner product formula uses the phase it sees opposite values. These opposite values cancel each other out in the calculation (Ockenga et al., 2023). The algorithm incorrectly calculates a very low overlap and drops the similarity score. Therefore the baseline method cannot reliably track continuous chord changes over time, specific in our project.

3.2 Spectral Line Shape Approach

Another approach focuses on the true spectral line shape of the frequency peaks. This method completely ignores the phase information of the sound wave. It instead calculates an exact spectral overlap score using the standard Lorentzian line shape.

3.2.1 The Lorentzian Factor and Visual Overlap

The Lorentzian factor uses spectral line shape. It calculates how much two frequency peaks intersect based purely on their distance and their width (Petrakis, 1967).

$$L(\Delta f) = \frac{1}{1 + (\frac{\Delta f}{\gamma})^2}$$

In this equation Δf is the difference between the two frequencies. The symbol γ is the width (= decay) of the peak. The L factor only calculates the visual overlap of the frequencies. It outputs a simple score where 1.0 means the frequencies are in the exact same location.

3.2.2 Adding Volume as a Weight

The Lorentzian factor only looks at the position of the frequencies. However the strength of the sound is also very important. A loud guitar string must have a bigger impact on the final score than a quiet noise.

To solve this the algorithm multiplies the calculated Lorentzian factor by the actual amplitudes of the resonances. This multiplication combines two important elements into one final score. The new score shows how well the frequencies match and how strong those frequencies actually are.

3.2.3 Fixed Height of Peak

If we look at the Lorentzian factor, the top part of the fraction is always 1. This is a fixed mathematical number. Because of this the absolute top of every frequency peak is forced to be exactly 1.0. A very narrow peak and a very wide peak will both hit this exact same maximum height.

In reality this creates a problem. The total energy of a sound wave must stay balanced (Müller, 2015). If a real sound wave becomes wider its peak should become lower to keep the overall energy the same. However our formula permanently locks the height at 1. By doing this the formula completely ignores how real energy works.

3.2.4 The Normalization Failure

To finalize the similarity metric the algorithm attempts to normalize the raw overlap score. The goal of this step is to limit the final answer to a interval between 0.0 and 1.0. However because the calculated area under our curve is mathematically broken the normalization simply does not work. When the algorithm divides the overlap by an incorrect physical power it produces wild and unpredictable numbers. The final score is not bounded to the 0.0 and 1.0 interval. Because of this failure the visual amplitude method becomes completely unusable for tracking continuous chord changes.

3.3 The Normalized Lorentzian approach

The final solution proposed in this thesis uses the normalized form of the Lorentzian function/Cauchy Distribution. In mathematics, a normalized distribution, such as the Cauchy distribution, means that the formula is designed so that the true area under the curve is exactly equal to one (Riley et al., 2006). Unlike the previous approach we discussed, this version does not force the peak to a fixed height. Instead it balances the height and the width so that the total calculated area always stays perfectly accurate. This true area is important because it represent the true area under the curve. Because we now have a reliable mathematical area we can calculate the exact overlap between different Lorentzian peaks. By using this correct area together with the power of the sound instead of the starting amplitude we solve the mathematical problems of the previous approaches. This final approach guarantees an algorithm for detecting chord transitions that is independent of phase.

3.3.1 Formula

The scientific version of the Lorentzian distribution is mathematically designed so that the total area under the curve is always exactly 1.0. To make sure this area stays perfectly constant the formula uses a multiplier. It divides the main equation by the mathematical constant π (Riley et al., 2006).

$$L(f) = \frac{1}{\pi} \left[\frac{\gamma}{(f - f_0)^2 + \gamma^2} \right]$$

This design makes it sure that the height and the width of the peak are always in perfect balance. If a resonance becomes very narrow, the formula automatically pushes the peak higher to keep the total area equal to 1. This balance is essential because it allows the algorithm to accurately represent the true energy of the wave.

3.3.2 Removing factor $\frac{1}{\pi}$

Our actual code uses a simpler version of the formula by removing the division by pi:

$$L = \frac{\gamma_{total}}{(\Delta f)^2 + \gamma_{total}^2}$$

Strictly speaking, removing the $\frac{1}{\pi}$ factor means the total area under the curve becomes π instead of exactly 1.0. However, the π constant is just a fixed scaling number that does not change the shape or location of the sound. When we calculate our final similarity score using the Cauchy Schwarz fraction, this π factor would appear on both the top and the bottom. Because of this, it cancels itself out. The final similarity score remains exactly the same whether we include π or not. By leaving it out, we save unnecessary calculations and make the algorithm much faster without losing any mathematical accuracy.

3.3.3 Adding Power as a Weight

The Lorentzian factor only calculates the overlap between two frequency peaks. However the actual strength of the sound is also very important. A loud sustained sound must have a bigger impact on the final similarity score than a quiet background noise. To achieve this the algorithm must multiply the raw Lorentzian factor by a weight.

We use the total power of the peak as this weight instead of the starting amplitude. Amplitude only measures the strength of a sound wave at the exact starting point where time is zero (Homer et al., 2024). This creates a bias. A sound wave might start with a massive initial amplitude but decay very quickly. If we only use amplitude as a weight this short sound would incorrectly dominate the score.

The true power of a tone represents the total energy over time (Müller, 2015). It perfectly combines the initial amplitude and the decay of the sound. By multiplying our Lorentzian factor with the true power of both peaks we ensure that loud and sustained sounds correctly weigh heavier in the final similarity score.

3.3.4 Raw Overlap and Double Factor Problem

To calculate the total similarity we first need to calculate the raw overlap score between the individual peaks. The logical step is to multiply the power of peak A and the power of peak B with the Lorentzian equations. Initially, we attempted to calculate two separate factors. One factor to check if peak A reaches the centre of peak B and another factor to check if peak B reaches the centre of peak A. This approach would use the following formula:

$$Overlap = Power_A \times Power_B \times \left(\frac{decay_A}{\Delta f^2 + decay_A^2} \right) \times \left(\frac{decay_B}{\Delta f^2 + decay_B^2} \right)$$

This calculation introduces a mathematical problem. When two narrow peaks are positioned a little apart, they fail to reach the exact centre frequency of each other. Because of this distance, both separate Lorentzian factors drop to a very small fraction. In mathematics when you multiply two extremely small fractions together the final result becomes exponentially smaller. This is conceptually very similar to the well-known vanishing gradient problem in artificial neural networks, where multiplying small values during backpropagation causes the signal to vanish almost completely (Hochreiter, 1998). Because our initial formula forced these two small fractions to multiply, the total similarity score vanished in the exact same way. This creates a false score because the wide edges of both sounds could be actually still overlap. We can see this vanishing problem as a punishment by the formula for small fractions.

3.3.5 The Convolution Solution

To solve this problem we must look at how sound waves actually mix. The correct mathematical way to calculate the shared area of two sounds is called convolution (Müller, 2015) (Tatum, 2017). Our sound peaks use the Cauchy that we will cover in the next section. In mathematics this specific curve is called the Cauchy distribution. In acoustics the exact same shape is known as the Lorentzian function. Because they are mathematically identical we will often use the name Lorentzian in this thesis. If you mix two of these shapes together the result is simply a new Cauchy/Lorentzian shape. Because of this mathematical rule we do not need to multiply two separate fractions. We only need to add their widths together. By adding the decay of peak A and the decay of peak B we create one perfect combined formula (Tatum, 2017):

$$L = \frac{decay_A + decay_B}{\Delta f^2 + (decay_A + decay_B)^2}$$

This single formula calculates the exact overlap between the two sounds. It looks at the total combined width so the score no longer drops to zero caused by the vanishing problem we talked in the previous section. Now we can calculate the true raw overlap score. We simply multiply this new factor with the power of both peaks:

$$\text{Overlap} = \text{Power}_A \times \text{Power}_B \times L$$

This final value perfectly represents the actual shared energy between the two frequencies. It is now ready for the final normalization step.

Interestingly, the baseline formula by Homer et al. (2024) already adds the decays together in the bottom part of the equation. At first, we tried multiplying the separate peaks, but this caused the vanishing problem. This problem made us realize exactly why Homer et al. (2024) adds the widths together. It is mathematically needed to prevent the shared energy from completely dropping to zero.

3.3.6 Cauchy-Schwarz Normalization

The raw overlap score is not a reliable metric on its own. It relies too heavily on the loudness of the audio. If a sound is very loud the raw score becomes artificially high. To fix this we must normalize the score.

We base this normalization on a fundamental mathematical law called the Cauchy-Schwarz inequality (Aldaz et al., 2015). This strict law states that the overlap between two different signals can never be larger than the overlap of a signal with itself.

Applying this mathematical law directly results in the standard cosine similarity formula. This is the exact same metric used in the baseline approach from Chapter 2 (Homer et al., 2024). However we now apply this standard formula to our new convolution-based overlap scores. The algorithm calculates three specific values: the shared overlap between window A and window B the self-overlap of window A and the self-overlap of window B. We use the following equation:

$$S(A, B) = \frac{\text{Overlap}(A, B)}{\sqrt{\text{Overlap}(A, A) \times \text{Overlap}(B, B)}}$$

By dividing the shared overlap by the square root of the two self-overlaps we make the score perfectly bounded. Because of the Cauchy-Schwarz inequality the top part can never be larger than the bottom part. Therefore the result will always fall exactly between 0.0 and 1.0. A score of 1.0 means the two audio windows are identical while a score of 0.0 means they share no frequencies.

Finally our code includes a safety check for silent audio. If an audio window is completely silent its self-overlap is exactly zero. Since division by zero is mathematically impossible the algorithm first checks the self-overlaps. If it finds a zero it safely skips the calculation and simply returns a score of 0.0.

4 Experimental Setup

The goal of this experiment is to find chord changes in audio files. We use the DRS software to extract the resonances from the sound (Homer et al., 2024).

The audio is divided into small time blocks called *windows*. The duration of such a block is called the *window size*. We apply our mathematical similarity score to compare the extracted resonances of two adjacent windows.

When the music stays the same the score is high. If a new chord starts the score drops heavily. We define this sudden drop as a *transient*. Finding these transients is how we locate the exact chord changes.

4.1 Dataset

To test the algorithm we divided the data into two categories. We started with a controlled artificial dataset and then moved to complex real world audio recordings.

4.1.1 Synthetic Audio

We created the artificial data using the software *Audacity* (noa, a). This provided a perfectly controlled environment without any background noise. The dataset consists of short structured audio files. For example a file might play exactly two seconds of a C-chord followed immediately by two seconds of an G-chord. Because we generated these files manually we know the absolute exact timestamp of every change. This precise knowledge is essential to validate if our algorithm detects the transitions at the correct moments. Besides the fact that we know where the changes are, you can also listen to the sound and validate your results.

We built these chords using square waves. The usage of square waves are important, so that we get the overtones of a tone. Because artificial waves do not naturally fade out their physical decay is essentially zero. As explained in the previous chapter a decay of zero would cause the Lorentzian formula to break. To solve this we added a very small artificial decay of 0.0001 to the synthetic data. This allowed the formula to work perfectly on these generated sounds.

4.1.2 Real Audio

After validating the mathematics with the synthetic data, we tested the system on real piano and guitar recordings. We divided this real world data into three increasing levels of complexity (Brady, 2021)

- Monophonic Data: Audio containing only single notes playing one after the other. For example, first the C tone and after that the D tone .
- Homophonic Data: Audio containing full chords where all notes stop and start at the exact same time. For example the C chord followed by the G chord, where all tones of the C chord die at the same time.
- Polyphonic Data: The most complex audio where notes overlap and continue. In this data some notes of a chord might continue playing while others change.

4.2 Evaluation Metrics

To measure how well the algorithm works we use the F1-score (noa, b). This metric balances *Precision* and *Recall*. Precision shows us how many of our detections are correct. Recall shows how many of the real chord changes we actually found. We calculate these metrics by comparing our algorithm results against the real changes in the dataset.

To decide if a similarity score indicates a chord change we use a fixed threshold. If the score of an audio window drops below 0.10 (which means less than 10 percent similarity) the algorithm registers it as a possible change.

However our sliding window method introduces a specific counting problem. When the chord changes, the sounds mix in the overlapping parts. Window 0 contains only the old chord. Window

1 sits exactly on the transition and contains a mix of both chords. Window 2 contains only the new chord.

Because the algorithm compares adjacent windows, both the transition to window 1 and the transition to window 2 will often cause the score to drop below the 0.10 threshold. This means one real change creates two or more separate detections in the system. The standard evaluation would count this as one True Positive and one False Positive. This would unfairly drop the F1-score.

To solve this problem we used a clustering technique. If the algorithm detects multiple scores below the threshold directly next to each other, we merge them together. The system counts this entire group of low scores as one possible change. This method is perfect for sudden changes but also handles gradual changes where a slow transition might stretch across three, four or even more consecutive windows.

Finally we verify this chosen timestamp against the real changes in the audio. Because our window size is exactly 0.09 seconds we enforce a strict error margin of ± 0.09 seconds. This creates a small valid period around the detection. If the real chord change happens anywhere inside this specific period we accept the detection as a valid True Positive.

5 Results and Analysis

In this we will chapter look at the results of our tests. We compare different mathematical methods that we talked about in the earlier chapters to see how well they find chord changes. We discuss two early methods that did not work for our project. Then we show the successful results of our final normalized Lorentzian approach. Finally we evaluate how well the algorithm works on different types of music using the F1-score.

5.1 Finding Real Changes

Before we can calculate the F1-score we need to know exactly where the real chord changes happen.

To find these exact times we open the audio in Audacity and zoom in on the sound waves. When we look very closely at the visual wave we can clearly see where a sound starts, where it fades out, and where the next sound begins.

Actually for monophonic and homophonic sounds you can still hear it. The music just stops and starts very cleanly. But for polyphonic sound it is much harder. If you do not play an instrument it is going to be very hard to detect this. But as I said, you can look at the visual wave.

5.2 The Limitation of the Phase Method

As we talked earlier about the method in the paper of Homer. This method had an other goal and was for that reason not useful for our project. We also see in the results that these results have no meaning.

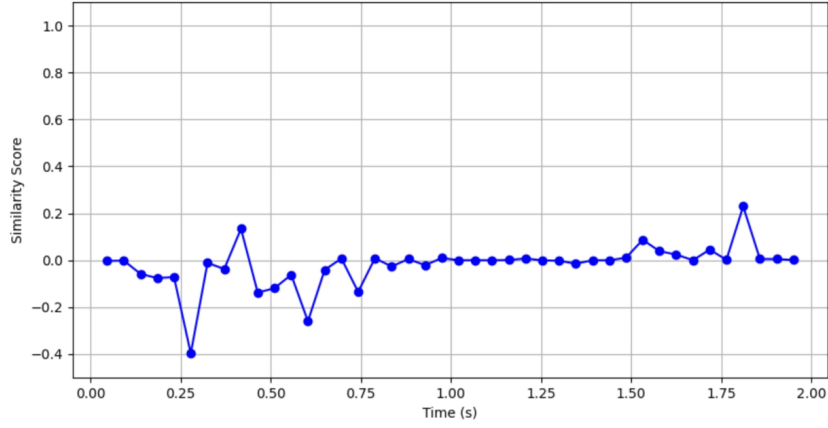


Figure 2: The similarity scores using the method by Homer et al. (2024). The values frequently drop below zero due to phase interference.

As you can see, the results are very often really close to zero and sometimes beneath zero. So our hypothesis about the phase being the problem is correct.

5.3 The Failed Unnormalized Lorentzian Method

We have also tested the spectral line shape method. As we explained in chapter 3 this method also failed. The problem was that the formula forced the peak height to exactly 1. This completely broke the true area beneath the curve.

Because the calculated power was wrong the final normalization step failed as well. Instead of giving a clean score between 0.0 and 1.0 the program just gave weird and unpredictable numbers. Because of this we could not use this method at all to find real chord changes. One thing we can actually agree is that this method was false, but was actually better then the one before. It is still unreliable but we can see some sort of dips sometimes in the results. Ofcourse since this methods mathematical base is not good, we can not rely on its results.

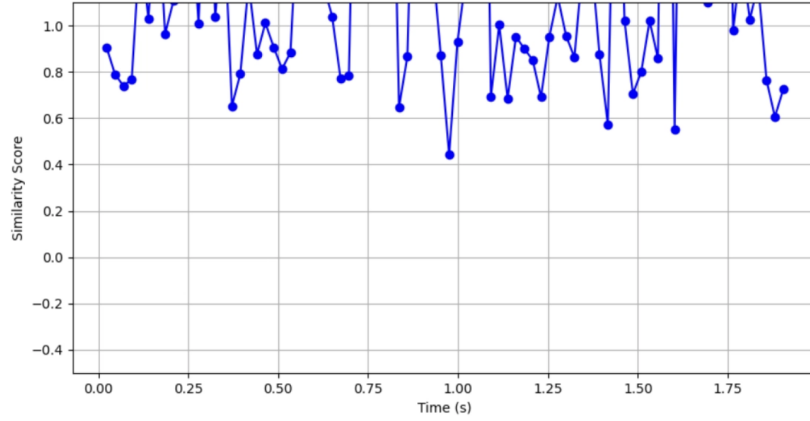


Figure 3: The similarity scores using the Unnormalized Lorentzian method. The values are unpredictable and fail to stay within the required 0.0 to 1.0 range.

As you can see, there is actually a dip at 1.0 seconds of the audio. But because we could not normalise, it was very hard to find a general threshold or a pattern for a relative threshold. Since it is not a reliable method, there was no need to calculate the F1-score for this method as well.

5.4 The Working Method

We already explained the normalized Cauchy distribution method in the previous chapter. The practical results of this third approach are very good.

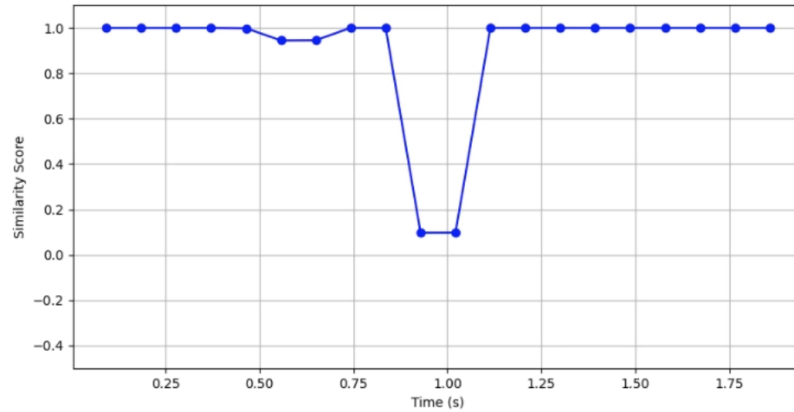


Figure 4: The similarity scores using the successful normalized method. The values stay perfectly between 0.0 and 1.0 with a clear dip at the chord change.

Figure 4 shows a perfectly stable score that always stays between 0.0 and 1.0. You can clearly see a massive dip exactly where the real chord changes. This immense visual contrast makes it incredibly easy to set a reliable detection threshold. We used this exact successful method to calculate all the final scores in the next sections.

6 Final F1-Score Results

Now we can look at the F1-scores of our working model. The F1-score combines precision and recall into one final grade. Precision indicates how many of the detected changes are actually real. Recall indicates how many of the total real changes the computer successfully found. We calculate the F1-score using this exact formula (noa, b):

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

We tested our algorithm on synthetic data and real audio.

6.1 Syntethic Data

- F1-Score: 0.99

As earlier told, the data files for this test is completely generated by Audacity. It has absolutely zero background noise and no physical decay. As I also mentioned earlier, we add a very small fracture of decay (0.0001) to let the data look like real data. The perfect score is exactly what we expected. It proves that our mathematics and our system worked perfect.

6.2 Monophonic Sound

For the monophonic audio we calculated a combined F1-score. The algorithm analyzed single melodies played on a real physical instrument. (Brady, 2021)

- Precision: 90.9%
- Recall: 71.4%
- F1-Score: 80.0%

This is a very good result. It proves that the program handles real instruments good. The high precision shows that almost every hit is true.

6.3 Homophonic Sound

For the homophonic audio we calculated a combined F1 score from various separate tests.

- Precision: 54.5%
- Recall: 80.0%
- F1-Score: 64.9%

The recall is very high. This means the program finds most of the real chord changes. The precision is lower, that was expected cause full chords contain many complex frequencies. The system sometimes thinks a messy fade out is actually a brand new chord. (Brady, 2021)

6.4 Polyphonic Sound

For the polyphonic audio we calculated a combined F1-score. This is the most difficult category. It contains multiple independent chords with constant tones that overlap several chords. (Brady, 2021)

- Precision: 50.0%
- Recall: 75.0%
- F1-Score: 60.0%

Even in this difficult audio the program still finds three quarters of all real chord changes. This proves that our normalization method is strong enough to handle complex overlapping sounds.

7 Conclusion

This project successfully created an algorithm to detect chord changes in audio. We evaluated three different mathematical approaches to solve this problem. The phase method and the unnormalized amplitude method both failed. The normalized Cauchy distribution method proved to be the absolute best solution. It ignores phase interference and calculates a highly stable similarity score. Our results show that the program performs exceptionally well on synthetic sounds and single melodies. The accuracy drops slightly for complex polyphonic music because overlapping frequencies hide the exact sound decay. The final scores clearly prove that our mathematical model is highly effective. This success opens the door for exciting new research. Future projects could focus on understanding the musical harmony space by clustering audio directly in the spectral domain. Future researchers can use our program to split a song into separate chords. They can then group these sounds by looking at their exact frequencies. This helps us see the real mathematics behind musical harmony. We can use pure computer data to explain exactly why certain notes sound good together.

References

Audacity  | Free Audio editor, recorder, music making and more!

fl_score.

(2016). Waves, Spectra and Sounds.

Aldaz, J. M., Barza, S., Fujii, M., and Moslehian, M. S. (2015). Advances in Operator Cauchy–Schwarz inequalities and their reverses. *Annals of Functional Analysis*, 6(3):275–295.

Brady, S. (2021). Texture. Book Title: Open Music Theory.

Brien, F. O. (2025). The Human Range of Hearing - Walker Hearing Clinics.

Hochreiter, S. (1998). The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6:107–116.

Homer, S. T., Harley, N., and Wiggins, G. A. (2024). Modelling of Musical Perception using Spectral Knowledge Representation. *Journal of Cognition*, 7(1):32.

Joutsenvirta, A. (2005). Basics of Acoustics – The pitch as a hertz value.

Müller, M. (2015). *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer International Publishing, Cham.

Ockenga, W., DeRose, J., and Greb, C. (2023). Phase Contrast and Microscopy.

Petrakis, L. (1967). Spectral line shapes: Gaussian and Lorentzian functions in magnetic resonance. *Journal of Chemical Education*, 44(8):432.

Riley, K. F., Hobson, M. P., and Bence, S. J. (2006). *Mathematical methods for physics and engineering*. Cambridge University Press, Cambridge, 3rd ed edition. OCLC: 569384420.

Tatum, J. (2017). 10.4: Combination of Profiles.

Wiggins, G. A. (2020). Creativity, information, and consciousness: The information dynamics of thinking. *Physics of Life Reviews*, 34-35:1–39.